Министерство науки и высшего образования РФ
АРЗАМАССКИЙ ПОЛИТЕХНИЧЕСКИЙ ИНСТИТУТ (ФИЛИАЛ)
ФЕДЕРАЛЬНОГО ГОСУДАРСТВЕННОГО БЮДЖЕТНОГО
ОБРАЗОВАТЕЛЬНОГО УЧРЕЖДЕНИЯ ВЫСШЕГО ОБРАЗОВАНИЯ «НИЖЕГОРОДСКИЙ
ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ ИМ. Р.Е.АЛЕКСЕЕВА»
(АПИ НГТУ)

Лабораторная работа по дисциплине «Логика и архитектура вычислительных средств»

Выполнила:
студентка гр. МА26ПМ
Фролова Д.А.

Проверил:
Комаров А.О.

Арзамас, 2026

СОДЕРЖАНИЕ

Введение 3

1 Требования к сервису 4

2 Архитектура системы 5

3 Взаимодействие сервисов 7

4 Хранилища и API 8

5 Паттерны проектирования 9

5.1 Chain of Responsibility 9

5.2 Strategy и Adapter 9

5.3 Repository, Factory и Circuit Breaker 9

6 Реализация и запуск 10

7 Тестирование и результаты 11

Заключение 14

Список литературных источников 15

Введение

Цель лабораторной работы - спроектировать и реализовать event-driven сервис антифрода для заказа концертных билетов. Сервис принимает событие покупки, оценивает вероятность мошенничества и формирует решение ALLOW либо REVIEW. В distributed-режиме сообщения передаются через Apache Kafka [1], оперативные признаки ведутся в Redis [7], решения сохраняются в PostgreSQL [6], а аналитическая сводка строится в ClickHouse [2].

Актуальность задачи определяется ботами, массовым выкупом билетов для перепродажи и повторным использованием устройств несколькими учётными записями. Для проверки подхода реализованы HTTP API на FastAPI [4], rule-based оценка и ML-модель LightGBM [5], а также отдельные consumer group для обработки событий. Учебный датасет синтетический и не содержит персональных данных.

1 Требования к сервису

Сервис должен принять заказ, проверить входные данные, рассчитать риск, вернуть решение и сохранить его для последующего запроса. В таблице 1.1 зафиксированы требования и границы учебной реализации.

Таблица 1.1 - Требования и данные

Категория	**Содержание**
Функциональные требования	Принять заказ, рассчитать риск, вернуть решение и причины срабатывания правил, сохранить решение и предоставить его по идентификатору заказа.
Нефункциональные требования	HTTP API, контейнерный запуск, воспроизводимое обучение модели, проверка входных данных и тестирование высокого и низкого риска.
Входные данные	Идентификаторы заказа, аккаунта, устройства и события; число билетов; сумма; число попыток; возраст аккаунта; связи устройства; отказы платежа; час.
Выходные данные	order_id, risk_score от 0 до 1, decision, список причин, версия модели и время расчёта.
Ограничения	Датасет синтетический и не содержит ПДн. Локальный режим использует in-memory адаптеры; distributed-режим развёрнут в Docker Compose [3].

Решение REVIEW направляет заказ на ручную проверку, а ALLOW разрешает дальнейшее оформление. Граница принятия решения настраивается в ML-адаптере; для исследуемой модели выбран порог 0,15.

2 Архитектура системы

Архитектура строится вокруг событий заказа. В local-режиме API использует in-memory адаптеры для разработки и unit-тестов. В контейнерном режиме отдельные сервисы выполняют обогащение, скоринг, транзакционное сохранение и запись аналитики. Схема контейнеров приведена на рисунке 2.1.

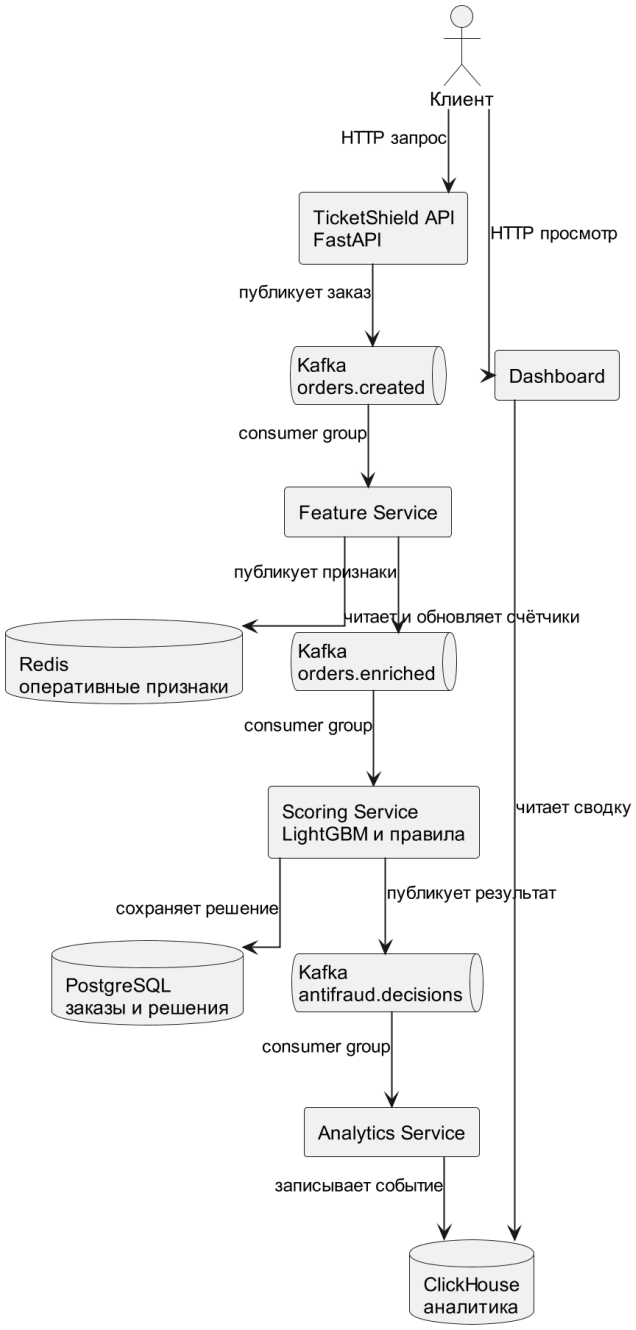


Рисунок 2.1 - Контейнерная архитектура TicketShield

Контейнеры имеют чётко разделённую ответственность. Сводка их интерфейсов приведена в таблице 2.1.

Таблица 2.1 - Ответственность контейнеров

Контейнер	**Ответственность**	**Интерфейс**
TicketShield API	Валидация заказа и публикация orders.created; в local-режиме выполняет синхронный скоринг.	REST /api/v1/orders/score
Kafka	Доставка заказов, обогащённых событий и решений между сервисами.	topics и consumer groups
Feature Service	Обогащение заказа счётчиками Redis и запись исходного заказа.	feature-service
Scoring Service	Расчёт риска, сохранение решения и публикация результата.	scoring-service
ClickHouse	Аналитика решений и данные dashboard.	SQL и /dashboard

3 Взаимодействие сервисов

После создания заказа API публикует событие в Kafka. Feature Service читает его в consumer group feature-service, дополняет счётчиками Redis и публикует обогащённое событие. Scoring Service рассчитывает риск, сохраняет решение в PostgreSQL и передаёт аналитическое событие. Analytics Service читает итоговый topic независимо, поэтому подключение нового потребителя не блокирует скоринг.

Перечень topic и контрактов сообщений приведён в таблице 3.1. Все сообщения сериализуются как UTF-8 JSON, что проверено отдельным тестом с русским текстом.

Таблица 3.1 - Kafka topics и группы потребителей

Topic	Producer	Consumer group	Формат сообщения
orders.created	TicketShield API	feature-service	UTF-8 JSON OrderEvent
orders.enriched	Feature Service	scoring-service	OrderEvent с признаками Redis
antifraud.decisions	Scoring Service	analytics-service	ScoreResponse с решением

Пример полезной нагрузки, который API принимает до публикации в topic orders.created, показан в листинге 1.

```
    {"order_id":"ord-demo-001","account_id":"acc-
100","device_id":"dev-77",
"ticket_count":6,"amount":42000,"purchase_attempts_10m":5,
"account_age_days":1,"device_accounts_24h":3}
```

Листинг 1 - Пример данных и команды запуска

4 Хранилища и API

PostgreSQL используется для транзакционных данных, Redis - для короткоживущих признаков, ClickHouse - для аналитических агрегатов. Назначение таблиц и ключей приведено в таблице 4.1.

Таблица 4.1 - Хранилища данных

Хранилище	Таблица или ключ	Назначение
PostgreSQL	orders и fraud_decisions	Заказы и решения; индексы по аккаунту, устройству, статусу и времени.
Redis	velocity:{account_id}:10m	Счётчик попыток покупки с TTL 10 минут.
Redis	device_accounts:{device_id}:24h	Множество аккаунтов устройства с TTL 24 часа.
ClickHouse	antifraud_events	Агрегаты решений и метрики dashboard по датам.

REST API содержит GET /health, POST /api/v1/orders/score, GET /api/v1/orders/{order_id}, GET /api/v1/dashboard/summary и страницу /dashboard. Pydantic-схема проверяет границы количества билетов, суммы и числовых признаков до передачи события в модуль скоринга. В distributed-режиме POST подтверждает публикацию статусом PENDING, а результат доступен по GET после обработки consumer group.

Контракт ответа API приведён в листинге 2.

```
    POST /api/v1/orders/score
Локальный режим: 201 +
{"decision":"REVIEW","risk_score":0.3653,...}
Distributed-режим: 201 +
{"order_id":"...","status":"PENDING"}
```

Листинг 2 - Пример данных и команды запуска

5 Паттерны проектирования

5.1 Chain of Responsibility

Паттерн применяется в app/scoring.py. Каждый объект правила самостоятельно проверяет одно условие: высокую частоту попыток, массовую покупку, свежий аккаунт или совместное использование устройства. RuleChain последовательно запускает правила, собирает срабатывания и суммирует веса. Поэтому добавление нового правила не изменяет API-контроллер.

5.2 Strategy и Adapter

Стратегия скоринга представлена rule baseline и LightGBM. Контроллер использует модель при наличии артефакта и переключается на правила при её отсутствии. LightGBMAdapter изолирует внешнюю библиотеку за методом predict_risk(event), поэтому замена модели не меняет контракт API.

5.3 Repository, Factory и Circuit Breaker

Интерфейс ScoreStore задаёт операции сохранения заказа и решения. Реализации InMemoryScoreStore и PostgresScoreStore имеют единый контракт. Factory-функция build_service собирает адаптеры по APP_MODE, а CircuitBreaker защищает публикацию orders.created: после трёх ошибок Kafka API временно возвращает HTTP 503. Такое разделение изолирует бизнес-логику от инфраструктуры.

6 Реализация и запуск

Проект реализован на Python 3.12. FastAPI отвечает за HTTP-слой, Pydantic - за схемы и валидацию, LightGBM - за расчёт риска, aiokafka - за producer и consumer Kafka, redis-py - за feature store, asyncpg - за PostgreSQL, clickhouse-connect - за аналитическое хранилище. Docker Compose поднимает Kafka, PostgreSQL, Redis, ClickHouse, API и три независимые consumer group.

Для воспроизводимого запуска достаточно выполнить команды из листинга 3. Конфигурация Docker Compose прошла проверку, а healthcheck не запускают прикладные контейнеры до готовности зависимостей.

```
    python -m pytest -q
docker compose up --build -d
python run_distributed_smoke.py
docker compose logs --tail=100 scoring-service
```

Листинг 3 - Пример данных и команды запуска

7 Тестирование и результаты

Для обучения сформирован воспроизводимый датасет из 2400 исторических событий: 1800 записей вошли в обучающую, а 600 - в тестовую выборку. Для сопоставления применён baseline на четырёх правилах. Сравнение метрик приведено в таблице 7.1.

Таблица 7.1 - Метрики моделей

Показатель	**Правила**	**LightGBM**
ROC-AUC	0,5660	0,6326
PR-AUC	0,0470	0,0879
Precision	0,0553	0,1000
Recall	0,4583	0,0833
F1	0,0987	0,0909
Доля REVIEW	0,3317	0,0333

По PR-AUC LightGBM превосходит baseline: 0,0879 против 0,0470. При выбранном пороге модель уменьшает долю заказов, передаваемых на ручную проверку, с 33,17 % до 3,33 %. Наиболее важными признаками стали amount, account_age_days и hour.

Набор автотестов проверяет API, ML-артефакт, валидацию, кодировку UTF-8, Circuit Breaker, локальный эквивалент pipeline и dashboard. Результаты тестовых сценариев приведены в таблице 7.2.

Таблица 7.2 - Автоматические тестовые сценарии

Сценарий	Ожидаемый результат	Статус
Высокий риск	Заказ с шестью билетами, пятью попытками и свежим аккаунтом получает REVIEW.	Покрыт тестом
Низкий риск	Обычный заказ получает ALLOW и доступен по GET-запросу.	Покрыт тестом
Валидация	ticket_count = 0 отклоняется с HTTP 422.	Покрыт тестом
ML-артефакт	Адаптер модели доступен при наличии файла модели.	Покрыт тестом
Kafka JSON	Русский текст проходит UTF-8 serialize и deserialize без искажений.	Покрыт тестом

Реальный результат локального запуска API в Swagger показан на рисунке 7.1. В нём отражены код HTTP 201, рассчитанный риск 0,3653, решение REVIEW и список правил.

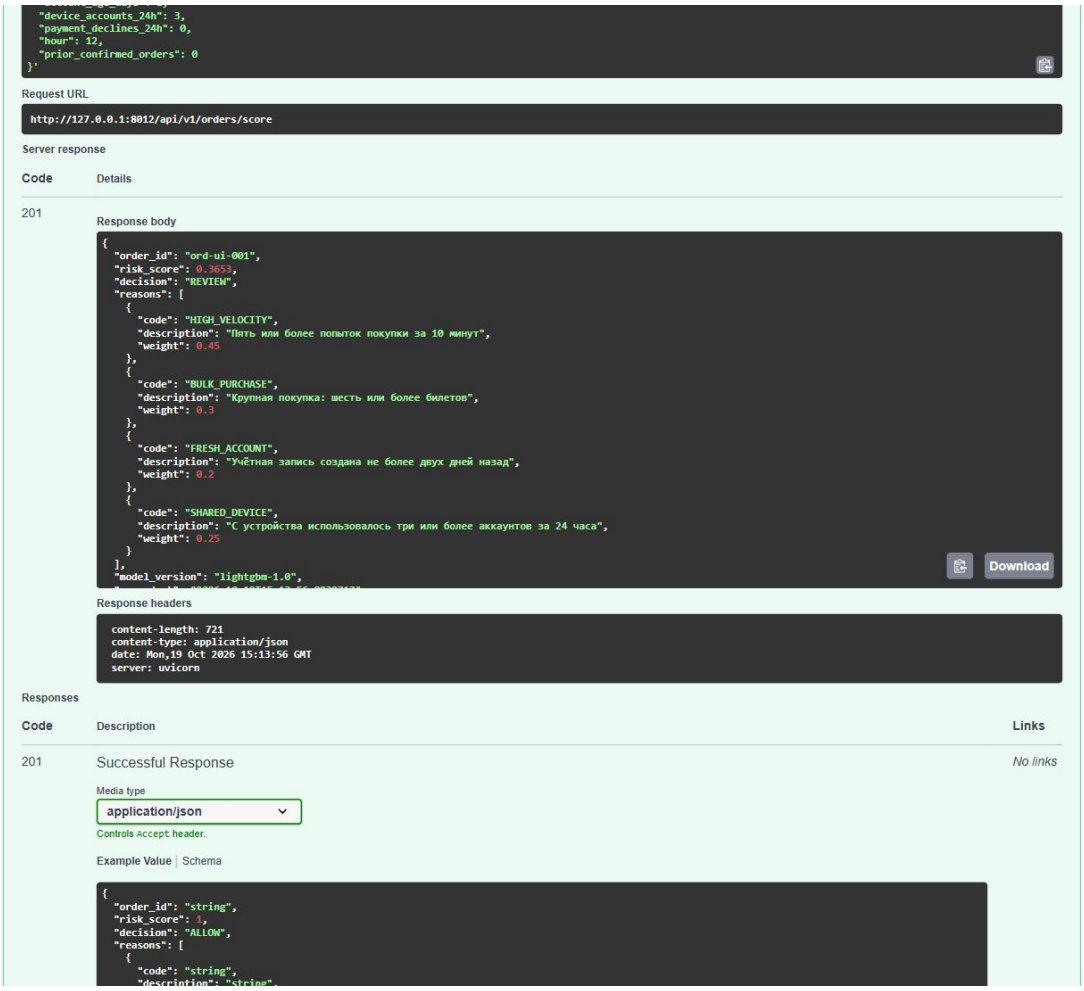


Рисунок 7.1 - Результат риск-скоринга в Swagger

Реальный dashboard распределённого стенда показан на рисунке 7.2. Он сформирован после прохождения API, Kafka, Redis, LightGBM, PostgreSQL и ClickHouse; сводка показывает четыре обработанных события и четыре решения REVIEW.

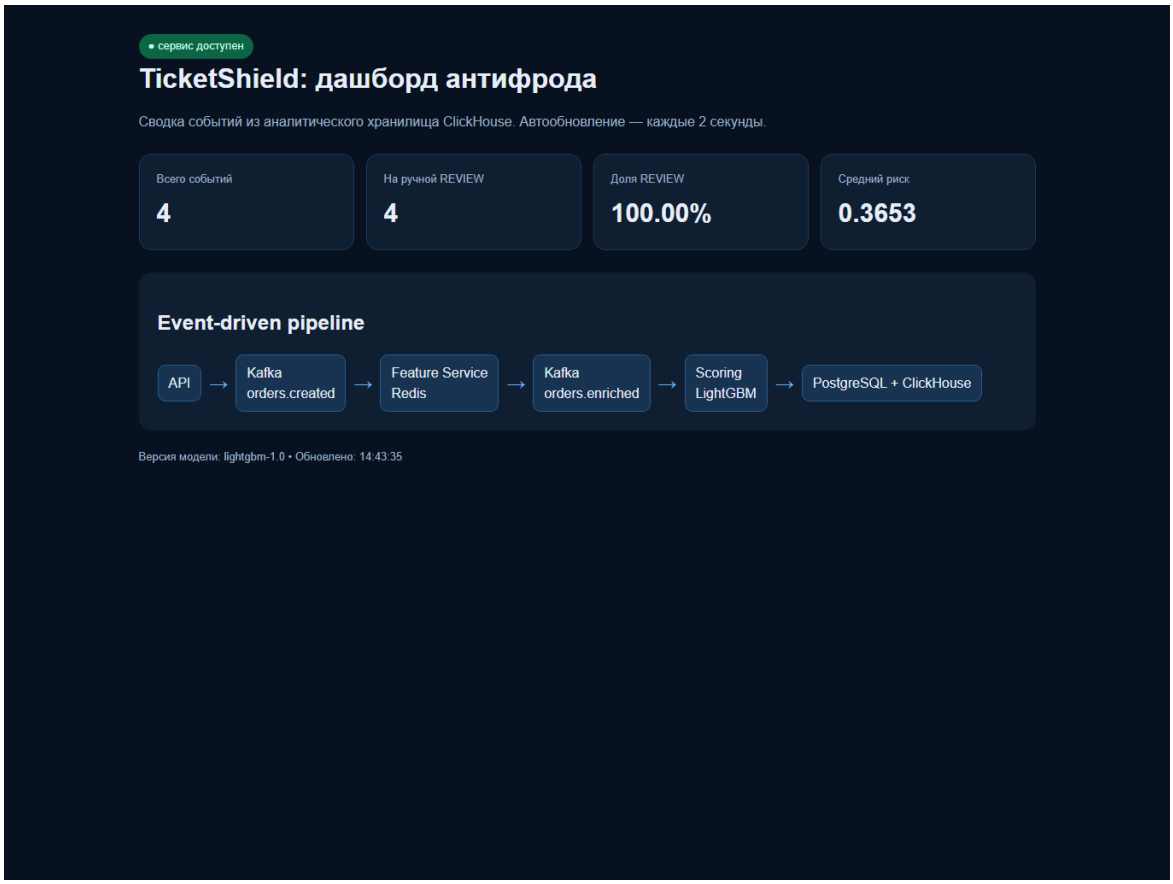


Рисунок 7.2 - Dashboard распределённого стенда

Контейнерный прогон выполнен фактически. Команда run_distributed_smoke.py получила PENDING от API, затем REVIEW с risk_score 0,3653 из PostgreSQL и актуальную сводку ClickHouse. В контейнере приложения выполнены 10 автоматических тестов: 10 passed, 1 warning.

Заключение

В лабораторной работе спроектирован и реализован сервис антифрода TicketShield. Реализация демонстрирует валидацию событий, гибридный rule-based и ML-скоринг, Kafka topics и consumer groups, Redis-признаки, Repository для PostgreSQL и ClickHouse-аналитику. Архитектурные решения позволяют выполнять локальную разработку с in-memory адаптерами и разворачивать те же контракты в Docker Compose без изменения API.

Проверка подтвердила корректность API, обработки сообщений и обновления аналитики. LightGBM повысил PR-AUC относительно rule baseline и сократил долю заказов, направляемых на ручную проверку. Разработанное решение соответствует цели лабораторной работы и может быть расширено новыми правилами, признаками и потребителями событий.

Список литературных источников

1. Apache Kafka Documentation [Электронный ресурс]. URL: https://kafka.apache.org/documentation/ (дата обращения: 20.10.2026).
2. ClickHouse Documentation [Электронный ресурс]. URL: https://clickhouse.com/docs (дата обращения: 20.10.2026).
3. Docker Docs [Электронный ресурс]. URL: https://docs.docker.com/ (дата обращения: 20.10.2026).
4. FastAPI Documentation [Электронный ресурс]. URL: https://fastapi.tiangolo.com/ (дата обращения: 20.10.2026).
5. LightGBM Documentation [Электронный ресурс]. URL: https://lightgbm.readthedocs.io/ (дата обращения: 20.10.2026).
6. PostgreSQL Documentation [Электронный ресурс]. URL: https://www.postgresql.org/docs/ (дата обращения: 20.10.2026).
7. Redis Documentation [Электронный ресурс]. URL: https://redis.io/docs/latest/ (дата обращения: 20.10.2026).